ECEN 468/719 Advanced Logic Design

Department of Electrical and Computer Engineering

Texas A&M University

Lab 8: Design of UART Transmitter

Objectives

In this lab, we will design a transmitter of Universal Asynchronous receiver/transmitter (UART) with Verilog and will use Design Analyzer for simulation. This module will be attached to our entire system later to transmit data to other devices or processors.

Introduction

The UART is a type of asynchronous receiver/transmitter, a computer hardware that translates data between parallel and serial forms. UARTs are commonly used with communication standards such as EIA RS-232, RS-422, or RS-485. The universal designation indicates that the data format and transmission speeds are configurable. Figure 1 shows a general description of communication between processors through a serial channel. Those processors communicate internally with parallel data to speed up and use a serial channel to communicate with other processors to reduce the number of wires, so decreasing the cost of hardware.

Figure 1. Communication over a serial channel

For this lab, a UART transmits 8-bit data without a parity bit. For transmission, the modem wraps this 8-bit word with a start-bit in the least significant bit (LSB) and a stop-bit in the most significant bit (MSB), resulting in the 10-bit word format shown in Figure 2. The first 9 data bits of the word are transmitted in sequence, beginning with the start-bit, with each bit being asserted at the serial line for one cycle (bit-time) of the modem clock. The stop-bit may assert for more than one clock.

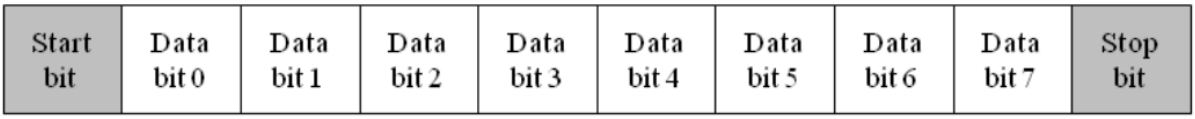


Figure 2. Data format for UART transmission

The simplified architecture of a UART is presented in Figure 3. It shows the signals used by a host processor to control the UART and to move data to and from a data bus in the host machine.

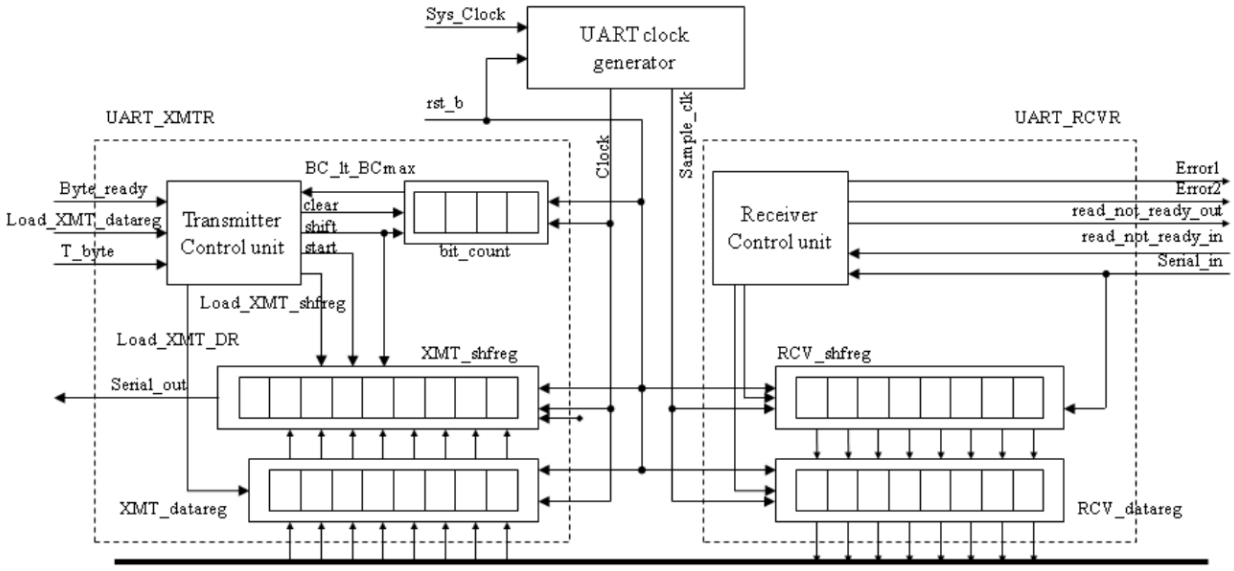


Figure 3. Block diagram of the UART

The input signals are provided by the host processor, and the output signal is the serial data stream. The architecture of the transmitter consists of a control unit, a data register (XMT_datareg), a data shift register (XMT_shfreg), and a status register (bit_count), which counts the bits that are transmitted.

The controller has the inputs (primary/external and status (from the datapath)) listed below. We note that the signal *Load_XMT_datareg* could be passed directly to the datapath unit; instead, we pass *Load_XMT_datareg* to the control unit and assert *Load_XMT_DR* conditionally when the state is *idle,* and the external signal *Load_XMT_datareg* is asserted. The status signal, *BC_lt_BCmax*, is asserted while bits are being sent, i.e., if *Bit_count* < *word_size* +1.

a. Load_XMT_datareg: assertion is state idle asserts *Load_XMT_DR*, which loads the content of the *Data_Bus* into *XMT_data_reg.*

b. Byte_ready: assertion causes *Load_XMT_shfreg* to assert, which loads the contents of *XMT_datareg* into *XMT_shfreg.*

c. T_byte: assertion initiates transmission of a byte of data, including the stop, start, and parity bits.

d. BC_lt_BCmax: indicates the status of the bit counter in the datapath unit.

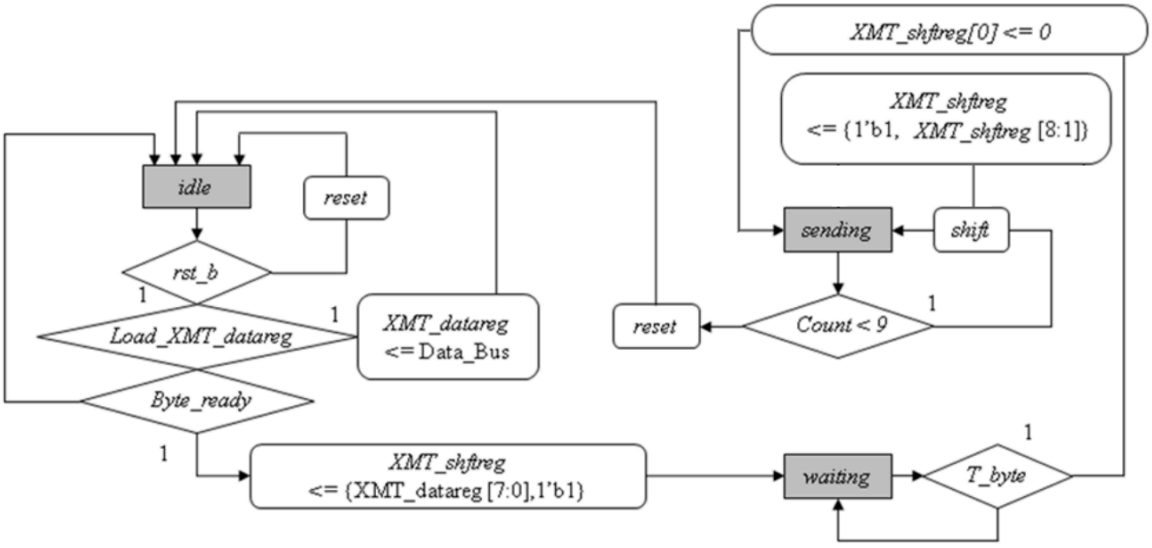


Figure 4. Algorithmic State Machine and Datapath Chart(ASMD) for the UART transmitter

The ASMD chart of the state machine controlling the transmitter is shown in Figure 4. The machine has three states: *idle*, *waiting*, and *sending*. When the active-low, synchronous reset signal *rst_b* is asserted, the machine enters *idle*, *bit_count* is flushed, and *XMT_shftreg* is loaded with 1s. In *idle*, if an active edge of *Clock* occurs while *Load_XMT_data_reg* is asserted by the external host, it will load *XMT_data_reg* with the contents of *Data_Bus*. The machine remains *idle* until the *start* is asserted to drop *XMT_shftreg*[0].

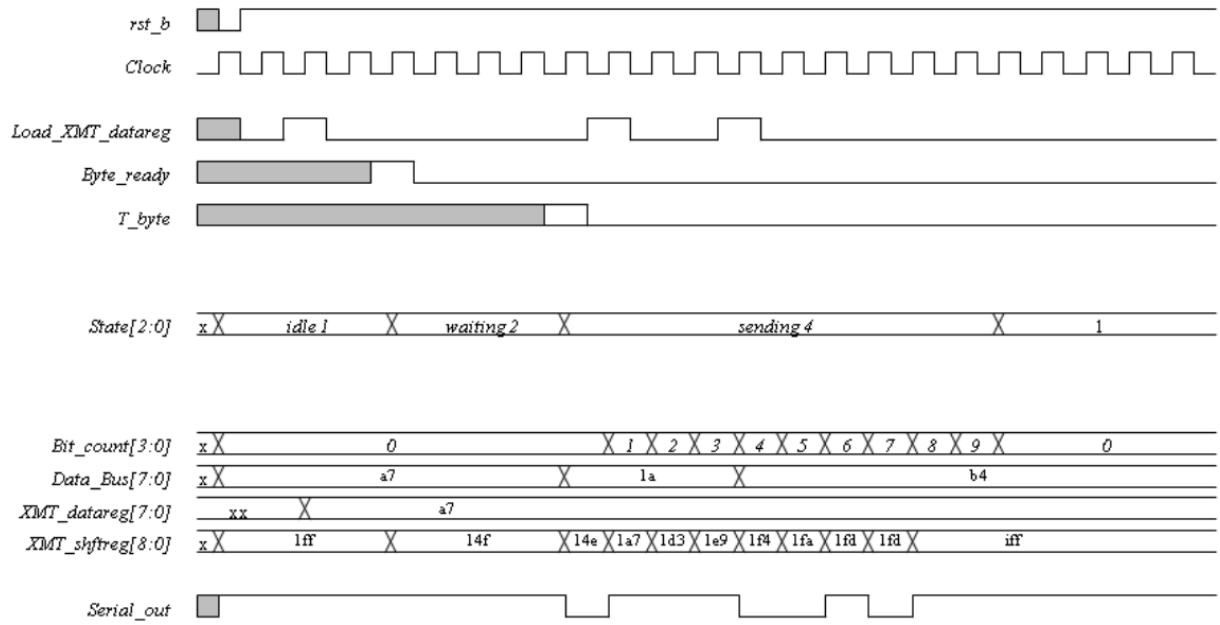


Figure 5. Waveforms of the 8-bit UART transmitter

Figure 5 shows an example of the transmission timing. You can check your result based on this timing graph.

User-Defined Primitives (UDP)

Verilog has built-in primitives like gates, transmission gates, and switches. If we need more complex primitives, Verilog provides UDP, or simply User Defined Primitives. Using UDP, we can model combinational logic and sequential logic. We can include timing information along with these UDP to model complete ASIC library models. **In this lab, we will design comparing block to generate signal BC_lt_BCmax using UDP. The signal BC_lt_BCmax should be '1' if bit_count is less than 9, otherwise '0'. Please put the output port as the first variable.**

```
// Example of UDP in Verilog.

      // This code shows how the UDP body looks like
```

```
primitive UDP_NAME (
      a, // Port a
      b, // Port b
      c, // Port c
);
// In/Out declaration
output a;
input b,c;

// UDP function code here
// A = B | C;
table
      // B C : A
      ? 1 : 1;
      1 ? : 1;
      0 0 : 0;
endtable
endprimitive
```

Implementation & Simulation

We will now implement the transmitter part of the UART design.

Please login to the Olympus server and create a working directory for this lab using the following commands.

```
## Create and navigate to the working directory.
mkdir -p $HOME/ECEN468/Lab8/src
cd $HOME/ECEN468/Lab8/src
```

Download the zip file (lab8_code.tar.gz) from the lab web page and extract it. In the extracted folders, you will find **UART_XMTR.v, Control_Unit.v, Data_Path.v, UART_tb.v, generic.sdb, osu018_stdcells.v and osu018_stdcells.db**. You will design modules in these files after you decompress the file. Copy them to the working directory.

You will implement your code in **"UART_XMTR.v", "Control_Unit.v", and "Data_Path.v".**

Once you complete the implementation, please verify the correctness of your design. We will first compile the design using the commands below.

Commands for reference:

```
load-ecen-468 (skip this line on machines in Zach 127)
source /opt/coe/synopsys/vcs/W-2024.09-SP2-4/setup.vcs.sh
vcs -full64 UART_tb.v
```

If it shows compile errors, please recheck your design and fix your code. If the compilation is complete, go to the next step to show the results.

```
./simv
```

Please take screenshots of the terminal output and include them in the report.

To view the graphical waveform of the simulation results, run the following command to invoke WaveView, and open "wave.dump" in WaveView

```
source /opt/coe/synopsys/wv/V-2023.12-4/setup.wv.sh
wv &
```

The graphical waveform is for debugging. You don't need to include it in the report.

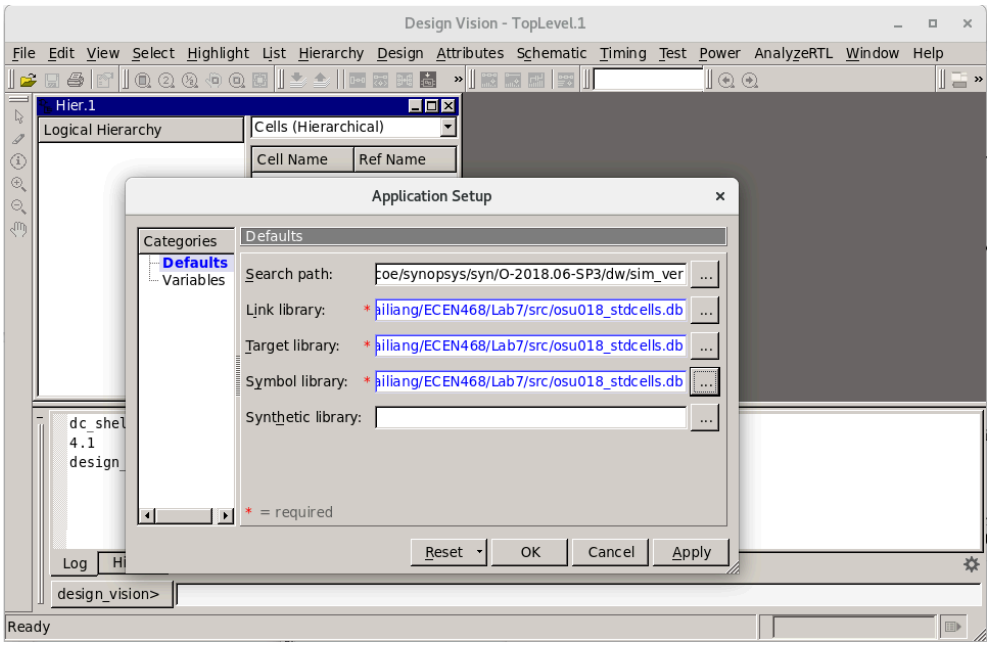
Once the verification is complete, we will use **Design Vision** to synthesize the design. For your convenience, please invoke Design Vision from a separate directory to isolate the intermediate files generated.

```
## Create and navigate to a separate working directory.
mkdir $HOME/ECEN468/Lab8/src/dv_Work
cd $HOME/ECEN468/Lab8/src/dv_Work
## Launch Design Vision
source /opt/coe/synopsys/syn/V-2023.12-SP1/setup.syn.sh
design_vision &
```

In Design Vision, first, we want to link the libraries.

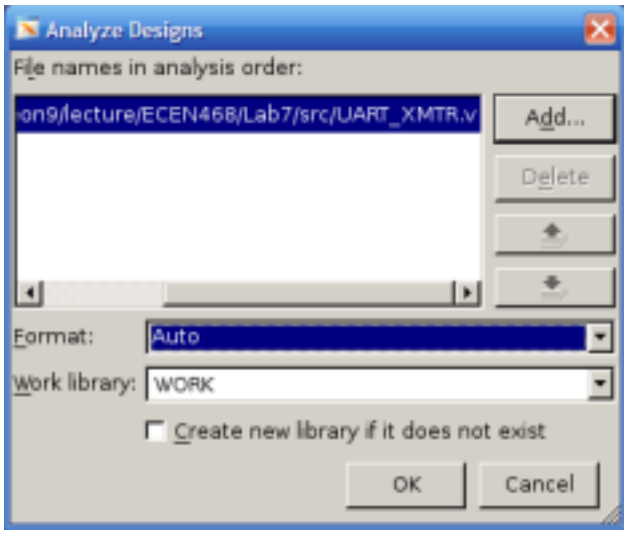
- Click on File -> Setup -> Defaults
- Add osu018_stdcells.db to "Link Library" (Remove all other library files)
- Add osu018_stdcells.db to "Target Library" (Remove all other library files)

- Add osu018_stdcells.db to “Symbol Library” (Remove all other library files)
- Click on OK



Now we want to *Analyze* the design. In *Analyze*, it reads VHDL or Verilog files, checks for proper syntax and synthesizable logic, and stores the design in some intermediate format.

- Click on File -> Analyze.
- In the pop-up window, add “UART_XMTR.v”, and click on OK.



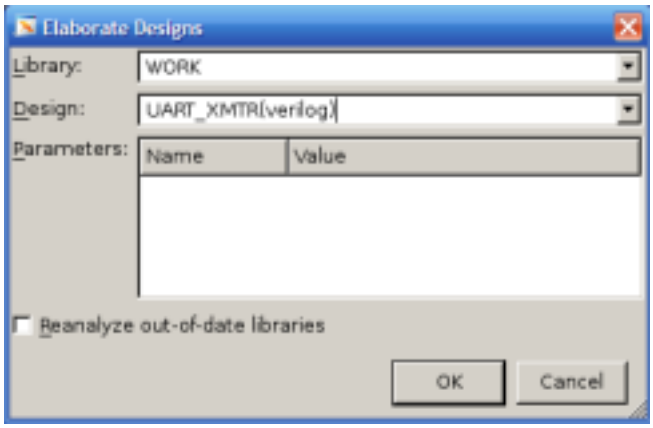
You will see an error message about the UDP design not being synthesizable.

To fix the error, use the line below to replace the UDP.

```
assign BC_lt_BCmax = (bit_count < word_size + 1);
```

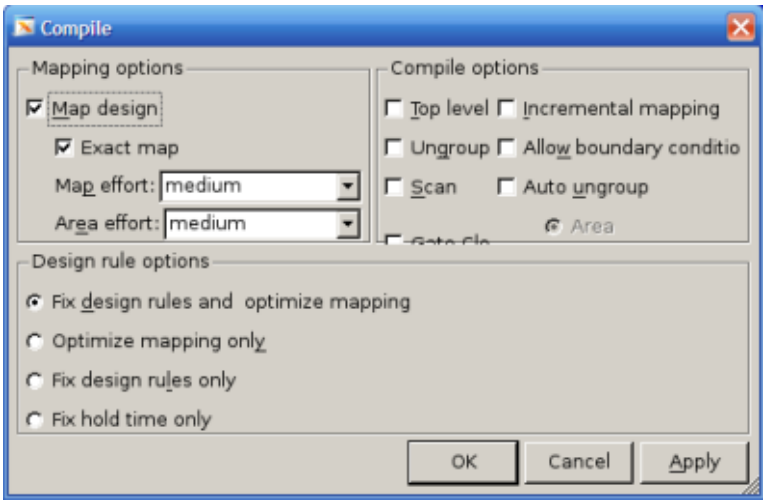
Elaborate Design

- Click on File -> Elaborate.
- In the pop-up window, type "work" in *Library*, and select UART_XMTR in *Design*.
- Click on OK.



Synthesize the module

Use ***Design -> Compile Design***. And Click *OK*.



Save the optimized Verilog netlist.

- Select *TOP* design (UART_XMTR)
- File -> Save as
- Please enter "***UART_gate.v***" as the file name and choose "Verilog" as the file type.
- Check the option "**Save All Designs in Hierarchy**".

Save the Standard Delay Format (SDF) by running the following command in the command window:

```
write_sdf UART.sdf
```

You may now close Design Vision.

Gate Simulation

a) Copy the Verilog netlist and sdf file to work folder.

```
cp ./UART_gate.v $HOME/ECEN468/Lab8/src/
cp ./UART.sdf $HOME/ECEN468/Lab8/src/
cd $HOME/ECEN468/Lab8/src/
```

We will use the previous testbench as a template to create a testbench for the gate simulation.

```
cp UART_tb.v UART_gate_tb.v
```

Please open the new testbench "UART_gate_tb.v" and do the following modifications:

- Make sure the following three lines are on the top of the file.

  ```
  `timescale 1ns/10ps
  `include "UART_gate.v"
  `include "osu018_stdcells.v"
  ```

- Remove the following line if it exists.

  ```
  `include "UART_XMTR.v"
  ```

- Make sure the following two lines are at the bottom of the file. (within the module entity)

  ```
  initial
        $sdf_annotate("UART.sdf", UART_XMTR_01);
  ```

- Change the name of the dump file in the following line in the test bench.

  ```
  $dumpfile ("wave_gate.dump");
  ```

Please make sure you have all the files listed below available in your current directory.
`UART_gate.v, UART.sdf, osu018_stdcells.v, UART_gate_tb.v`

Once we have the files ready, we can run the following command to start the simulation.

```
source /opt/coe/synopsys/vcs/W-2024.09-SP2-4/setup.vcs.sh
vcs -full64 UART_gate_tb.v
```

If no error occurs, do the next step.

```
./simv
```

Please take a screenshot of the simulation output, and include it in the report.

Submission

Please only submit one PDF file containing the following items:

UART:

1. Screenshots of the simulation output after running ./simv.
2. Justification of the correctness of the results.
3. Screenshots of the code “UART_XMTR.v”, “Control_Unit.v”, and “Data_Path.v”.

UART Gate:

1. Screenshots of the simulation output after running ./simv.
2. Justification of the correctness of the results.